Open in Colab

Floating Point Arithmetic

A number is called *floating point number* if it contains a `.`

In [1]:
```
12.3
```

Out[1]: 12.3

In [2]:
```
.1
```

Out[2]: 0.1

You can also use *scientific notation* when writing floating point numbers:

In [3]:
```
1.2354e3
```

Out[3]: 1235.4

An integer can be written as a floating point number by adding `.` after it.

In [4]:
```
12.
```

Out[4]: 12.0

Use `+` for addition, `-` for subtraction, `*` for multiplication and `**` for exponentation like integer arithmetic.

In [5]:
```
6.4*(1.25-12.)**5.
```

Out[5]: -918802.76875

When doing integer arithmetic if any of the operands is a floating point number or the result can be a floating point number then *floating point arithmetic* is performed.

Floating point arithmetic is very similar to integer arithmetic but uses different algorithms for computation.

Warning: Floating point arithmetic sometimes gives surprising results. For example:

In [6]:
```
0.1 * 3
```

Out[6]: 0.30000000000000004

Therefore when working on floating point numbers you should know that the result may not be exact.

For (true) division use: `/`

In [7]:
```
15.3 / 4.2
```

Out[7]: 3.642857142857143

The (true) division is always done using floating point arithmetic. Even if the result is integer.

In [8]:
```
12 / 3
```

Out[8]:
```
4.0
```

Conversion Between Integers and Floating Point Numbers

Between integers and floating point numbers, opearations are always done with floating point artihmetic and integrs are automatically converted to floating point numbers.

In [9]:
```
12 + 13.
```

Out[9]:
```
25.0
```

Comparision between floating point numbers is like integer arithmetic.

In [10]:
```
2. == 2 < 2.1
```

Out[10]:
```
True
```

Unlike integers floating point numbers can **not** be arbitrary big.

In [11]:
```
2.0**1023
```

Out[11]:
```
8.98846567431158e+307
```

Tu get some floating point number `number` rounded to `n` digits use `round(number,ndigits=n)`

In [12]:
```
round(12.34567, ndigits=3)
```

Out[12]:
```
12.346
```

Complex Number Arithmetic

Python also supports complex number arithmetic. Any complex number of the form $a + bi \in \mathbb{C}$ can be represented by `a+bj` in Python.

In [13]:
```
((3 + 1j) * 2) ** 2
```

Out[13]:
```
(32+24j)
```

To get the real and imaginary part of a number use `.real` and `.imag` properties.

In [14]:
```
(2 + 3j).real
```

Out[14]:
```
2.0
```

In [15]:
```
(2 + 3j).imag
```

Out[15]:
```
3.0
```

To get the absolute value of a complex number use `abs()` function.

In [16]:
```
abs(3+4j)
```

Out[16]: 5.0